Agent 365 AI Teammates Hands-On Lab

By the end of this lab, you will have:

- Created or reused an Agent 365 AI Teammate blueprint.
- Published the agent manifest to the tenant catalog.
- Deployed the Python backend to Azure Container Apps with `azd`.
- Registered the Container Apps `/api/messages` endpoint on the blueprint.
- Enabled agentic authentication so Teams messages are accepted by the backend.
- Verified the live endpoint and checked logs for incoming Teams activity.

The lab assumes you are using the repository from inside the VS Code dev container. The dev container already includes the required command-line tools, including `az`, `azd`, `a365`, PowerShell, Python, and the project virtual environment. Do not spend workshop time installing tools unless a validation step fails.

Before attendees start

The facilitator should confirm these tenant and Azure prerequisites before the workshop:

- The tenant is enrolled in the Agent 365 preview program.
- Attendees have the required Entra role, usually Agent ID Developer.
- Attendees have access to the target Azure subscription.
- The tenant has the one-time `Agent 365 CLI` app registration configured.
- Microsoft 365 tool service principals have been provisioned by an admin.
- An Azure OpenAI or Azure AI Foundry resource and deployment are available.
- The repository opens successfully in the VS Code dev container.

Keep secrets out of Git

Generated Agent 365 state files can contain client secrets. They should remain local only.

The repository should ignore these files:

```
a365.generated.config.json
a365.generated.config.*.json
.env
.azure/
```

Before pushing a branch, verify that no generated config file is tracked:

```
git ls-files 'a365.generated.config*.json'
git log --oneline --all -- 'a365.generated.config*.json'
```

Both commands should produce no output.

Step 1: Open the dev container

Open the repository in VS Code and select **Reopen in Container** when prompted. After the terminal opens, start from the repository root:

```
cd /workspaces/a365-maf
```

Quickly verify the expected tools are available:

```
a365 --version
az version -o table
azd version
python --version
pwsh --version
```

If these commands work, skip all installation instructions from the longer setup guide.

Step 2: Set lab variables

Choose a unique agent base name. Use lower-case letters, numbers, and hyphens. The name should be short enough to fit Azure naming limits.

```
export AGENT_NAME="<your-unique-agent-name>"
export AZURE_LOCATION="southeastasia"
export AZURE_SUBSCRIPTION_ID="<your-subscription-id>"
```

Example:

```
export AGENT_NAME="sithu-a365-02"
export AZURE_LOCATION="southeastasia"
export AZURE_SUBSCRIPTION_ID="69ecbfc6-bfbb-4fa4-9933-a5ec21627a3f"
```

Step 3: Sign in

Sign in to Azure CLI and Azure Developer CLI from inside the dev container:

```
az login --allow-no-subscriptions
az account set --subscription "$AZURE_SUBSCRIPTION_ID"
azd auth login
```

Confirm the active tenant and subscription:

```
az account show --query "{user:user.name, tenantId:tenantId, subscriptionId:id}" -o
json
```

Step 4: Validate Agent 365 prerequisites

Run the Agent 365 requirements check:

```
a365 setup requirements
```

The current dev container CLI validates the active Azure authentication context and does not accept `--agent-name` on the `requirements` subcommand. If you see `Unrecognized command or argument '--agent-name'`, rerun the command without that flag.

Continue only when the command reports no failures. A Frontier preview warning can be acceptable in the lab if the facilitator has already confirmed tenant enrollment.

Step 5: Create the AI Teammate blueprint

Run the AI Teammate setup command:

```
a365 setup all --aiteammate --agent-name "$AGENT_NAME"
```

For this lab path, the Agent 365 setup creates or reuses the blueprint and configures permissions. The Python backend is deployed later with `azd`.

If the CLI reports that the blueprint already exists, continue. The setup commands are idempotent and reuse resources by display name.

After setup completes, confirm the generated blueprint ID exists locally:

```
python3 -c "import json; print(json.load(open('a365.generated.config.json'))
['agentBlueprintId'])"
```

Step 6: Review and publish the manifest

Open `manifest/manifest.json` and update the attendee-facing details:

- `name.short`
- `name.full`
- `description.short`
- `description.full`
- `developer.name`
- `developer.websiteUrl`
- `developer.privacyUrl`
- `developer.termsOfUseUrl`
- `version`

Keep this value unchanged for this preview manifest:

```
"manifestVersion": "devPreview"
```

Publish the manifest:

```
a365 publish
```

Step 7: Create the azd environment

Create an `azd` environment for the backend deployment:

```
azd env new "$AGENT_NAME"
azd env set AZURE_LOCATION "$AZURE_LOCATION"
azd env set AZURE_SUBSCRIPTION_ID "$AZURE_SUBSCRIPTION_ID"
azd env set AZURE_RESOURCE_GROUP "rg-$AGENT_NAME"
```

If the environment already exists, select it instead:

```
azd env select "$AGENT_NAME"
```

If the workshop uses a shared Azure OpenAI or Azure AI Foundry resource, set the expected values now. Adjust these values to match the facilitator-provided resource:

```
azd env set AZURE_OPENAI_ACCOUNT_NAME "terenceaifoundry-resource"
azd env set AZURE_OPENAI_RESOURCE_GROUP "rg-admin-terenceaifoundry"
azd env set AZURE_OPENAI_ENDPOINT "https://terenceaifoundry-
resource.openai.azure.com"
azd env set AZURE_OPENAI_DEPLOYMENT "gpt-5.4"
azd env set AZURE_OPENAI_API_VERSION "preview"
```

Step 8: Deploy once to create the public endpoint

Deploy the backend with no blueprint credentials yet. This first pass creates the Container App and produces the public FQDN.

```
azd up --no-prompt
```

Get the deployed endpoint:

```
FQDN=$(azd env get-values | grep AGENT_FQDN | cut -d= -f2 | tr -d '"')
echo "https://$FQDN/api/messages"
```

Verify the health endpoint:

```
curl "https://$FQDN/api/health"
```

Expected response:

```
{"status": "ok", "agent_type": "AgentFrameworkAgent", "agent_initialized": true}
```

Step 9: Register the backend endpoint with the blueprint

Bind the Container Apps messaging endpoint to the Agent 365 blueprint:

```
a365 setup blueprint --agent-name "$AGENT_NAME" --endpoint-only \
  --messaging-endpoint "https://$FQDN/api/messages"
```

The command should report that the endpoint was registered successfully.

Step 10: Enable agentic authentication

Feed the blueprint ID, tenant ID, and blueprint client secret into the same `azd` environment.

```
CLIENT_ID=$(python3 -c "import json;
print(json.load(open('a365.generated.config.json'))['agentBlueprintId'])")
TENANT_ID=$(python3 -c "import json; print(json.load(open('a365.config.json'))
['tenantId'])")
SECRET=$(python3 -c "import json;
print(json.load(open('a365.generated.config.json'))
['agentBlueprintClientSecret'])")

azd env set BLUEPRINT_CLIENT_ID "$CLIENT_ID"
azd env set BLUEPRINT_TENANT_ID "$TENANT_ID"
azd env set BLUEPRINT_CLIENT_SECRET "$SECRET"

unset SECRET
```

> [!NOTE] Some older instructions use `azd env set --secret`. The dev container version of `azd` used for this lab does not support that flag. Use `azd env set BLUEPRINT_CLIENT_SECRET "$SECRET"`. This stores the value in local `azd` state under `.azure/`, which must stay ignored by Git.

Redeploy so the Container App receives the agentic auth environment variables:

```
azd up --no-prompt
```

Step 11: Verify the deployed auth configuration

Confirm the Container App is running and configured for the current blueprint:

```
RG=$(azd env get-values | awk -F= '/^AGENT_RESOURCE_GROUP=/{gsub(/"/,"",$2); print
$2}')
APP=$(azd env get-values | awk -F= '/^AGENT_CONTAINER_APP_NAME=/{gsub(/"/,"",$2);
print $2}')

az containerapp show -g "$RG" -n "$APP" \
  --query "{runningStatus:properties.runningStatus,
latestReadyRevisionName:properties.latestReadyRevisionName,
env:properties.template.containers[0].env[?
name=='CLIENT_ID'||name=='TENANT_ID'||name=='AUTH_HANDLER_NAME'||name=='USE_AGENTIC
_AUTH']}" \
  -o json
```

The `CLIENT_ID` should match the blueprint ID in `a365.generated.config.json`.

Probe the endpoints again:

```
curl "https://$FQDN/api/health"
curl -i "https://$FQDN/api/messages"
```

The health endpoint should return `200`. The unauthenticated `/api/messages` probe should return `401`, which is expected because Teams sends an authorized POST.

Step 12: Create and test an agent instance

Complete these steps in the browser:

1. Open the Microsoft 365 admin center.
2. Publish or approve the custom agent package for the tenant.
3. In Teams, find the published agent.
4. Create or request an instance.
5. Wait for the agent user account to appear in Teams.
6. Start a chat with the new agent user and send `hello`.

Every instance created from the same blueprint posts to the same backend endpoint. The Python app uses the incoming activity metadata to distinguish the human user and the agent instance.

Step 13: Watch logs during the Teams test

Tail the Container App logs while sending a Teams message:

```
az containerapp logs show -g "$RG" -n "$APP" --tail 120 --follow
```

Useful success signals include:

```
POST /api/messages HTTP/1.1" 200
Validating agent and setting up context
tenant_id=
agent_id=
Got it
```

Stop the log stream with `Ctrl+C`.

Fast troubleshooting

If the Teams chat does not respond, check these items in order.

Messages never reach the container

Run:

```
az containerapp logs show -g "$RG" -n "$APP" --tail 120 | grep '/api/messages' || true
```

If there is no `POST /api/messages`, re-register the endpoint:

```
a365 setup blueprint --agent-name "$AGENT_NAME" --endpoint-only \
  --messaging-endpoint "https://$FQDN/api/messages"
```

Messages reach the container but return 401

Compare the deployed `CLIENT_ID` with the blueprint ID:

```
python3 -c "import json; print(json.load(open('a365.generated.config.json'))['agentBlueprintId'])"

az containerapp show -g "$RG" -n "$APP" \
  --query "properties.template.containers[0].env[?name=='CLIENT_ID'].value" \
  -o tsv
```

If the values differ, rerun Step 10 and Step 11.

The agent starts but fails while answering

Check recent errors:

```
az containerapp logs show -g "$RG" -n "$APP" --tail 200 | grep -E 'Error|Exception|AADSTS|Failed|Invalid' || true
```

Common causes are missing Azure OpenAI permissions, incorrect model deployment name, or MCP consent problems.

Cleanup after the lab

Only run cleanup if the facilitator confirms the resources are no longer needed.

Delete Azure resources created by `azd`:

```
azd down --purge
```

Do not run blueprint cleanup unless you intentionally want to remove the Agent 365 blueprint from the tenant.

```
a365 cleanup blueprint --agent-name "$AGENT_NAME"
```